Common Problems

Uber

Real-time Updates · Multi-step Processes

By Evan King · Updated Feb 15, 2026 · hard · Asked at: +4

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

▷ Watch Now

Understanding the Problem

🚙 **What is Uber?**

Uber is a ride-sharing platform that connects passengers with drivers who offer transportation services in personal vehicles. It allows users to book rides on-demand from their smartphones, matching them with a nearby driver who will take them from their location to their desired destination.

Functional Requirements

1. Start your interview by defining the functional and non-functional requirements. For user facing applications like this one, functional requirements are the "Users should be able to..." statements whereas non-functional defines the system qualities via "The system should..." statements.
2. Prioritize the top 3 functional requirements. Everything else shows your product thinking, but clearly note it as "below the line" so the interviewer knows you won't be including them in your design. Check in to see if your interviewer wants to move anything above the line or move anything down. Choosing just the top 3 is important to ensuring you stay focused and can execute in the limited time window.

Core Requirements

1. Riders should be able to input a start location and a destination and get a fare estimate.
2. Riders should be able to request a ride based on the estimated fare.
3. Upon request, riders should be matched with a driver who is nearby and available.
4. Drivers should be able to accept/decline a request and navigate to pickup/drop-off.

Below the line (out of scope)

1. Riders should be able to rate their ride and driver post-trip.
2. Drivers should be able to rate passengers.
3. Riders should be able to schedule rides in advance.
4. Riders should be able to request different categories of rides (e.g., X, XL, Comfort).

Non-Functional Requirements

Core Requirements

1. The system should prioritize low latency matching (< 1 minutes to match or failure)
2. The system should ensure strong consistency in ride matching to prevent any driver from being assigned multiple rides simultaneously
3. The system should be able to handle high throughput, especially during peak hours or special events (100k requests from same location)

Below the line (out of scope)

1. The system should ensure the security and privacy of user and driver data, complying with regulations like GDPR.
2. The system should be resilient to failures, with redundancy and failover mechanisms in place.
3. The system should have robust monitoring, logging, and alerting to quickly identify and resolve issues.
4. The system should facilitate easy updates and maintenance without significant downtime (CI/CD pipelines).

Functional Requirements

1. Riders request estimated fare
2. Riders except fare to get matched with a driver
3. Driver can accept/deny and navigate to pickup/drpoff

---------- Out of Scope ----------

- Rating drivers/riders
- Scheduling rides in advance
- Support different ride types (X, XL, etc)

Non-functional Requirements

1. Low latency ride matching (< 1 minute)
2. Strong consistency for matching
4. Handle surges (100k from same loc. @ same time)

---------- Out of Scope ----------

- GDPR
- Backups
- CI/CD
- Fault tolerance
- Monitoring

Uber Requirements

> Adding features that are out of scope is a "nice to have". It shows product thinking and gives your interviewer a chance to help you reprioritize based on what they want to see in the interview. That said, it's very much a nice to have. If additional features are not coming to you quickly, don't waste your time and move on.

The Set Up

Planning the Approach

Before you move on to designing the system, it's important to start by taking a moment to plan your strategy. Fortunately, for these common users facing product-style questions, the plan should be straightforward: build your design up sequentially, going one by one through your functional requirements. This will help you stay focused and ensure you don't get lost in the weeds as you go. Once you've satisfied the functional requirements, you'll rely on your non-functional requirements to guide you through the deep dives.

Defining the Core Entities

I like to begin with a broad overview of the primary entities. At this stage, it is not necessary to know every specific column or detail. We will focus on the intricacies, such as columns and

fields, later when we have a clearer grasp. Initially, establishing these key entities will guide our thought process and lay a solid foundation as we progress towards defining the API.

To satisfy our key functional requirements, we'll need the following entities:

1. **Rider**: This is any user who uses the platform to request rides. It includes personal information such as name and contact details, preferred payment methods for ride transactions, etc.
2. **Driver**: This is any users who are registered as drivers on the platform and provide transportation services. It has their personal details, vehicle information (make, model, year, etc.), and preferences, and availability status.
3. **Fare**: This entity represents an estimated fare for a ride. It includes the pickup and destination locations, the estimated fare, and the estimated time of arrival. This could also just be put on the ride object, but we'll keep it separate for now but there is no right or wrong answer here.
4. **Ride**: This entity represents an individual ride from the moment a rider confirms a fare estimate and requests a ride, all the way until its completion. It records all pertinent details of the ride, including the identities of the rider and the driver, vehicle details, state, the planned route, the actual fare charged at the end of the trip, and timestamps marking the pickup and drop-off.
5. **Location**: This entity stores the real-time location of drivers. It includes the latitude and longitude coordinates, as well as the timestamp of the last update. This entity is crucial for matching riders with nearby drivers and for tracking the progress of a ride.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.

Core Entities

- Driver
- Rider
- Fare
- Ride
- Location

Uber Core Entities

Now, let's proceed to design our system, tackling each functional requirement in sequence. This step-by-step approach will help us maintain focus and manage scope effectively, ensuring a cohesive build-up of the system's architecture.

> As you move onto the design, your objective is simple: create a system that meets all functional and non-functional requirements. To do this, I recommend you start by satisfying the functional requirements and then layer in the non-functional requirements afterward. This will help you stay focused and ensure you don't get lost in the weeds as you go.

API or System Interface

The API for retrieving a fare estimate is straightforward. We define a simple POST endpoint that takes in the user's current location and desired destination and returns a Fare object with the estimated fare and eta. We use POST here because we will be creating a new Fare entity in the database.

```
POST /fare -> Fare
Body: {
  pickupLocation,
  destination
}
```

Request Ride Endpoint: This endpoint is used by riders to confirm their ride request after reviewing the estimated fare. It initiates the ride matching process by signaling the backend to find a suitable driver, thus creating a new ride object.

```
POST /rides -> Ride
Body: {
  fareId
}
```

Note that at this point in the flow, we match them with a driver who is nearby and available. However, this is all happening in the backend, so we don't need to explicitly list out an endpoint for this.

Update Driver Location Endpoint: Before we can do any matching, we need to know where our drivers are. This endpoint is used by drivers to update their location in real-time. It is called periodically by the driver client to ensure that the driver's location is always up to date.

```
POST /drivers/location -> Success/Error
Body: {
        lat, long
    }

- note the driverId is present in the session cookie or JWT and not in the body or path pa
```

Always consider the security implications of your API. I regularly see candidates passing in data like `userId`, `timestamps`, or even `fareEstimate` in the body or query parameters. This is a red flag as it shows a lack of understanding of security best practices. Remember that you can't trust any data sent from the client as it can be easily manipulated. User data should always be passed in the session or JWT, while timestamps should be generated by the server. Data like `fareEstimate` should be retrieved from the database and never passed in by the client.

Accept Ride Request Endpoint: This endpoint allows drivers to accept a ride request. Upon acceptance, the system updates the ride status and provides the driver with the pickup location coordinates.

```
PATCH /rides/:rideId -> Ride
Body: {
  accept/deny
}
```

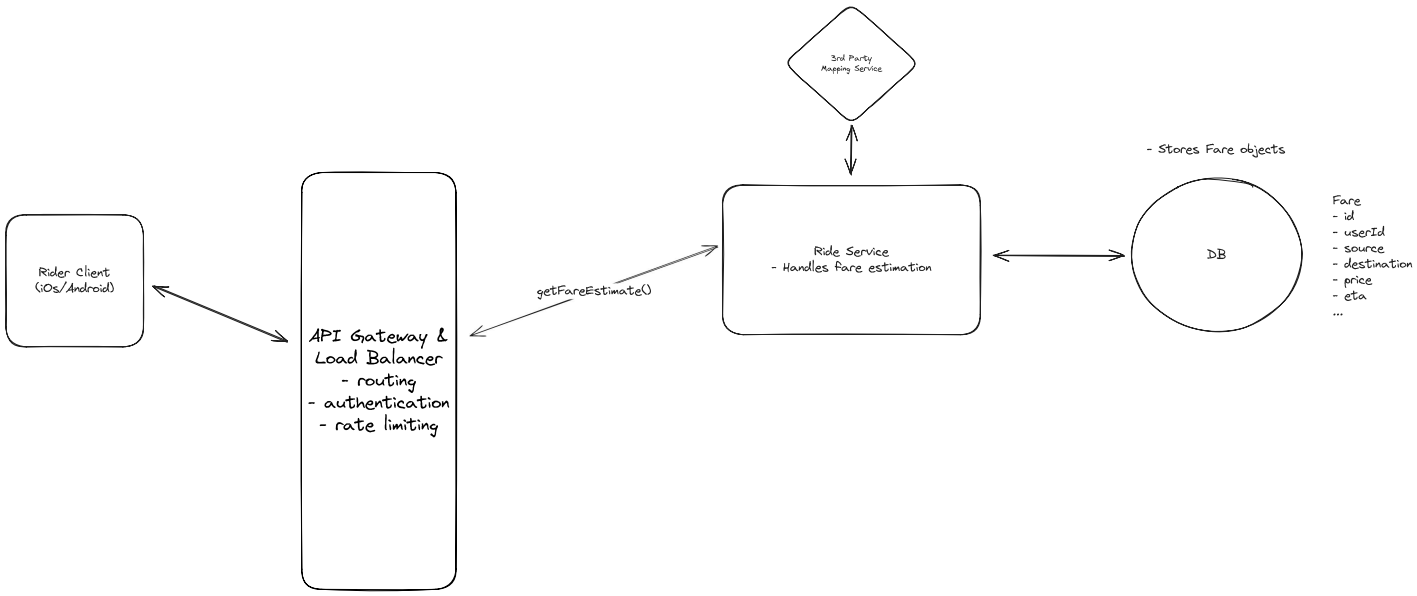
The `Ride` object would contain information about the pickup location and destination so the client can display this information to the driver.

High-Level Design

1) Riders should be able to input a start location and a destination and get an estimated fare

The first thing that users will do when they open the app to request a ride is search for their desired destination. At this point, the client will make a request to our service to get an estimated price for the ride. The user will then have a chance to request a ride with this fare or do nothing.

Lets lay out the necessary components for communicating between the client and our microservices, adding our first service, "Ride Service" which will handle fare estimations



Uber Simple Fare Estimation

The core components necessary to fulfill fare estimation are:

1. **Rider Client**: The primary touchpoint for users is the Rider Client, available on iOS and Android. This client interfaces with the system's backend services to provide a seamless user experience.
2. **API Gateway**: Acting as the entry point for client requests, the API Gateway routes requests to the appropriate microservices. It also manages cross-cutting concerns such as authentication and **rate limiting**.

3. **Ride Service**: This microservice is tasked with managing ride state, starting with calculating fare estimates. It interacts with third-party mapping APIs to determine the distance and travel time between locations and applies the company's pricing model to generate a fare estimate. For the sake of this interview, we abstract this complexity away.
4. **Third Party Mapping API**: We use a third-party service (like Google Maps) to provide mapping and routing functionality. It is used by the Ride Service to calculate the distance and travel time between locations.
5. **Database:** The database is, so far, responsible for storing Fare entities. In this case, it creates a fare with information about the price, eta, etc.

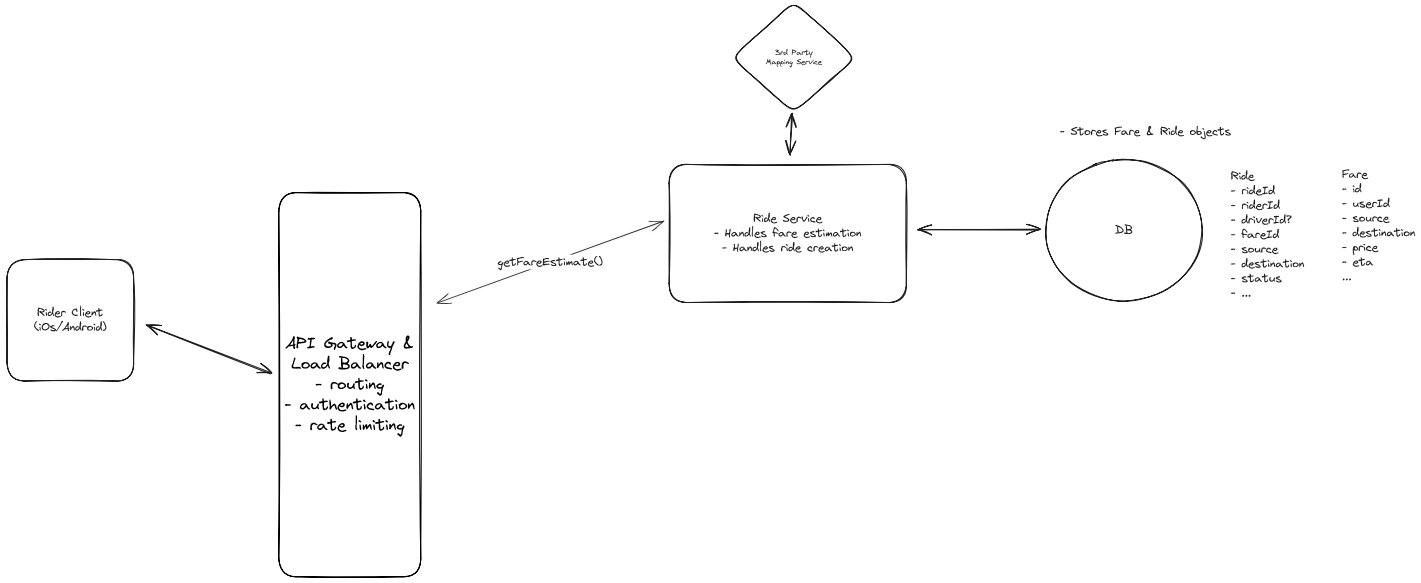
Let's walk through exactly how these component interact when a rider requests a fare estimate.

1. The rider enters their pickup location and desired destination into the client app, which sends a POST request to our backend system via `/fare`
2. The API gateway receives the request and handles any necessary authentication and rate limiting before forwarding the request to the Ride Service.
3. The Ride Service makes a request to the Third Party Mapping API to calculate the distance and travel time between the pickup and destination locations and then applies the company's pricing model to the distance and travel time to generate a fare estimate.
4. The Ride Service creates a new Fare entity in the Database with the details about the estimated fare.
5. The service then returns the Fare entity to the API Gateway, which forwards it to the Rider Client so they can make a decision about whether accept the fare and request a ride.

2) Riders should be able to request a ride based on the estimated fare

Once a user reviews the estimated fare and ETA, they can request a ride. By building upon our existing design, we can extend it to support ride requests pretty easily.

We don't need to add any new services at all, we just need to add a Ride table to our Database.



Riders should be able to request a ride based on the estimated fare

Then, when a request comes in, this is how we handle it.

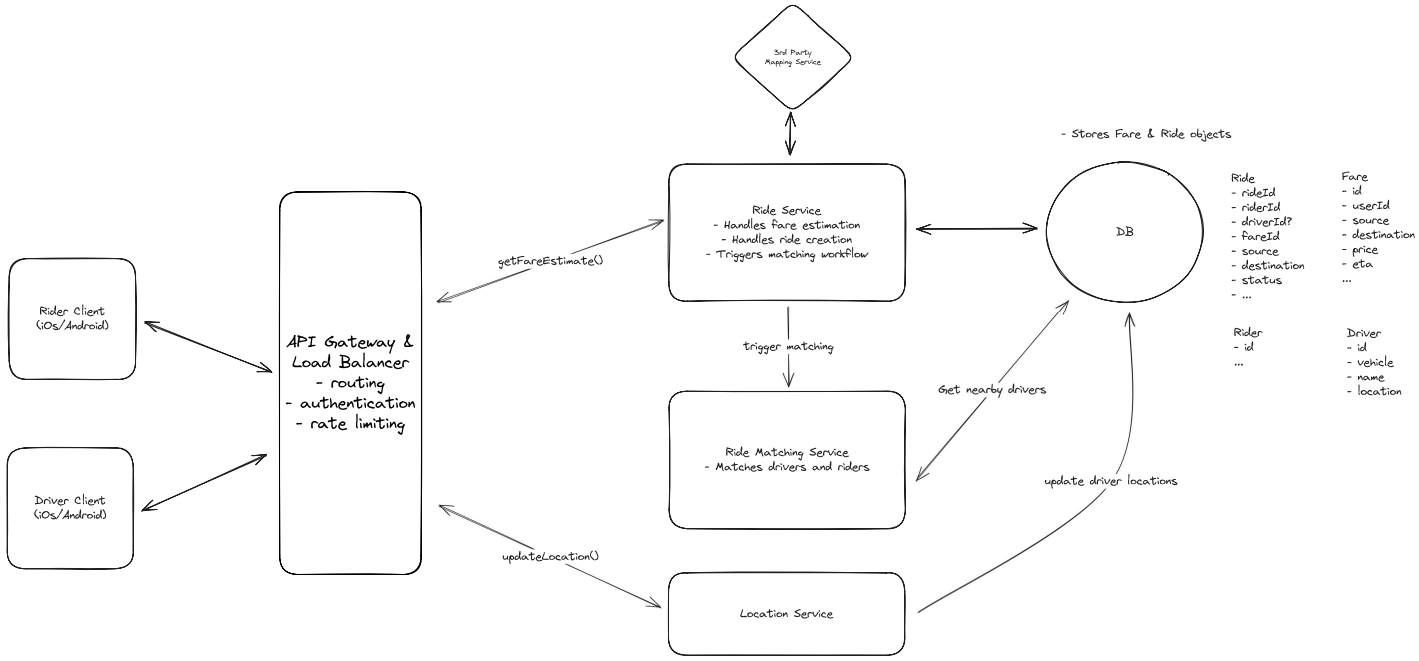
1. The user confirms their ride request in the client app, which sends a POST request to our backend system with the id of the Fare they are accepting.
2. The API gateway performs necessary authentication and rate limiting before forwarding the request to the Ride Service.
3. The Ride Service receives the request and creates a new entry in the Ride table, linking to the relevant Fare that was accepted, and initializing the Ride's status as `requested`.
4. Next, it triggers the matching flow so that we can assign a driver to the ride (see below)

3) Upon request, riders should be matched with a driver who is nearby and available

Now we need to introduce some new components in order to facilitate driver matching.

1. **Driver Client**: In addition to the Rider Client, we introduce the Driver Client, which is the interface for drivers to receive ride requests and provide location updates. The Driver Client communicates with the Location Service to send real-time location updates.
2. **Location Service**: Manages the real-time location data of drivers. It is responsible for receiving location updates from drivers, storing this information in the database, and providing the Ride Matching Service with the latest location data to facilitate accurate and efficient driver matching.

3. **Ride Matching Service**: Handles incoming ride requests and utilizes a sophisticated algorithm (abstracted away for the purpose of this interview) to match these requests with the best available drivers based on proximity, availability, driver rating, and other relevant factors.



Upon request, riders should be matched with a driver who is nearby and available

Let's walk through the sequence of events that occur when a user requests a ride and the system matches them with a nearby driver:

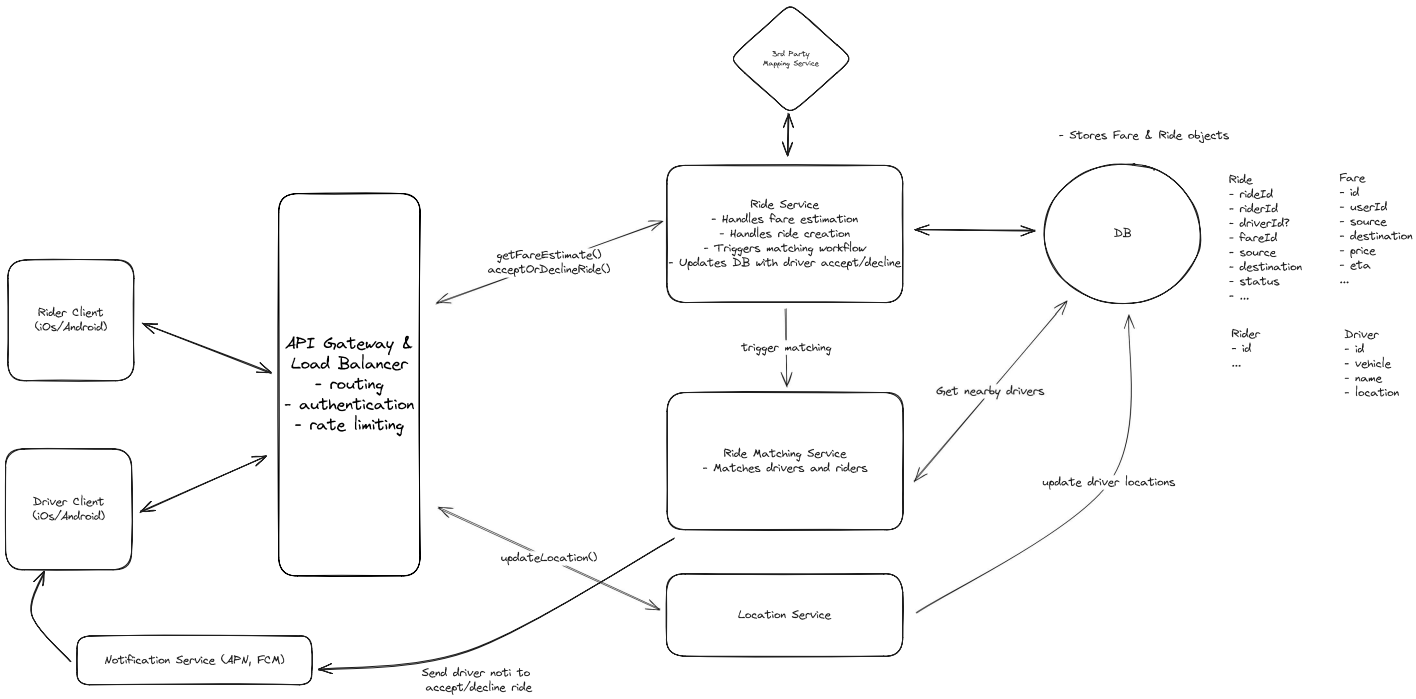
1. The user confirms their ride request in the client app, which sends a POST request to our backend system with the ID of the fare they are accepting.
2. The API gateway performs necessary authentication and rate limiting before forwarding the request to the Ride Service.
3. The Ride Service creates a ride object as mentioned above, and then forwards the request to the Ride Matching Service to trigger the matching workflow (we'll discuss ways to make this more robust later, keeping it simple for now).
4. Meanwhile, at all times, drivers are sending their current location to the location service, and we are updating our database with their latest location lat & long so we know where they are.
5. The matching workflow then uses these updated locations to query for the closest available drivers in an attempt to find an optimal match.

4) Drivers should be able to accept/decline a request and navigate to pickup/drop-off

Once a driver is matched with a rider, they can accept the ride request and navigate to the pickup location.

We only need to add one additional service to our existing design.

1. **Notification Service**: Responsible for dispatching real-time notifications to drivers when a new ride request is matched to them. It ensures that drivers are promptly informed so they can accept ride requests in a timely manner, thus maintaining a fluid user experience. Notifications are sent via APNs (Apple Push Notification service) and FCM (Firebase Cloud Messaging) for iOS and Android devices, respectively.



Uber Simple Driver Accept

Let's walk through the sequence of events that occur when a driver accepts a ride request and completes the ride:

1. After the Ride Matching Service determines the ranked list of eligible drivers, it sends a notification to the top driver on the list via APNs or FCM.
2. The driver receives a notification that a new ride request is available. They open the Driver Client app and accept the ride request, which sends a PATCH request to our backend system with the `rideID`. a) If they decline the ride instead, the system will send a notification to the next driver on the list.

3. The API gateway receives the requests and routes it to the Ride Service.
4. The Ride Service receives the request and updates the status of the ride to "accepted" and updates the assigned driver accordingly. It then returns the pickup location coordinates to the Driver Client.
5. With the coordinates in hand, the Driver uses on client GPS to navigate to the pickup location.

Pattern: Real-time Updates

Interviewer looking for a push notification to drivers? Our realtime updates pattern breakdown walks through the options from long-polling to SSE to Websockets.

Learn This Pattern

Potential Deep Dives

With the core functional requirements met, it's time to dig into the non-functional requirements via deep dives. These are the main deep dives I like to cover for this question.

> The degree to which a candidate should proactively lead the deep dives is a function of their seniority. For example, it is completely reasonable in a mid-level interview for the interviewer to drive the majority of the deep dives. However, in senior and staff+ interviews, the level of agency and ownership expected of the candidate increases. They should be able to proactively look around corners and identify potential issues with their design, proposing solutions to address them.

1) How do we handle frequent driver location updates and efficient proximity searches on location data?

Managing the high volume of location updates from drivers and performing efficient proximity searches to match them with nearby ride requests is a difficult task, and our current high-level design most definitely does not handle this well. There are two main problems with our current design that we need to solve:

1. **High Frequency of Writes**: Given we have around 10 million drivers, sending locations roughly every 5 seconds, that's about 2 million updates a second! Whether we choose something like DynamoDB or PostgreSQL (both great choices for the rest of the system), either one would either fall over under the write load, or need to be scaled up so much that it becomes prohibitively expensive for most companies.
2. **Query Efficiency**: Without any optimizations, to query a table based on lat/long we would need to perform a full table scan, calculating the distance between each driver's location and the rider's location. This would be extremely inefficient, especially with millions of drivers. Even with indexing on lat/long columns, traditional B-tree indexes are not well-suited for multi-dimensional data like geographical coordinates, leading to suboptimal query performance for **proximity searches**. This is essentially a non-starter.

> For DynamoDB in particular, 2M writes a second of ~~100 bytes at on-demand pricing (~~$1.25 per million WRUs) would cost you over $200k a day. Learn more about DynamoDB and its limitations here

So, what can we do to address these issues?

Bad Solution: Direct Database Writes and Proximity Queries

Good Solution: Batch Processing and Specialized Geospatial Database

Great Solution: Real-Time In-Memory Geospatial Data Store

2) How can we manage system overload from frequent driver location updates while ensuring location accuracy?

High-frequency location updates from drivers can lead to system overload, straining server resources and network bandwidth. This overload risks slowing down the system, leading to delayed location updates and potentially impacting user experience. In most candidates original design, they have drivers ping a new location every 5 seconds or so. This follow up

question is designed to see if they can intelligently reduce the number of pings while maintaining accuracy.

Great Solution: Adaptive Location Update Intervals

Don't neglect the client when thinking about your design. Many candidates get in a habit of drawing a small client box and moving on. In many cases, we need client side logic to improve the efficiency and scalability of our system. As you saw, we can reduce the number of pings by using ondevice sensors and algorithms to determine the optimal interval for sending location updates. Similarly, in the case of a file upload service, the client is responsible for chunking and compression.

3) How do we prevent multiple ride requests from being sent to the same driver simultaneously?

We defined consistency in ride matching as a key non-functional requirement. This means that we only request one driver at a time for a given ride request AND that each driver only receives one ride request at a time. That driver would then have 10 seconds to accept or deny the request before we move on to the next driver if necessary. If you've solved **Ticketmaster** before, you know this problem well -- as it's almost exactly the same as ensuring that a ticket is only sold once while being reserved for a specific amount of time at checkout.

Bad Solution: Application-Level Locking with Manual Timeout Checks

Good Solution: Database Status Update with Timeout Handling

Great Solution: Distributed Lock with TTL

4) How can we ensure no ride requests are dropped during peak demand periods?

During peak demand periods, the system may receive a high volume of ride requests, which can lead to dropped requests. This is particularly problematic during special events or holidays when demand is high and the system is under stress. We also need to protect against the case where an instance of the Ride Matching Service crashes or is restarted, leading to dropped rides.

Bad Solution: First-Come, First-Served with No Queue

Great Solution: Queue with Dynamic Scaling

5) What happens if a driver fails to respond in a timely manner?

Our system works great when the drivers either accept or deny the ride request, but what if the drop their phone in the passenger seat and take a break? How do we ensure that the ride request continues to be processed? Ideally we'd want the system to move on to the next driver if the current driver doesn't respond in a timely manner.

Pattern: Multi-step Processes

Human-in-the-loop processes are a big signal of when to use the Multi-Step Processes pattern. In fact, Uber is the original author of the open source project Cadence which gave birth to the leading durable execution framework Temporal, built specifically for use-cases like this one.

Learn This Pattern

Good Solution: Delay queue

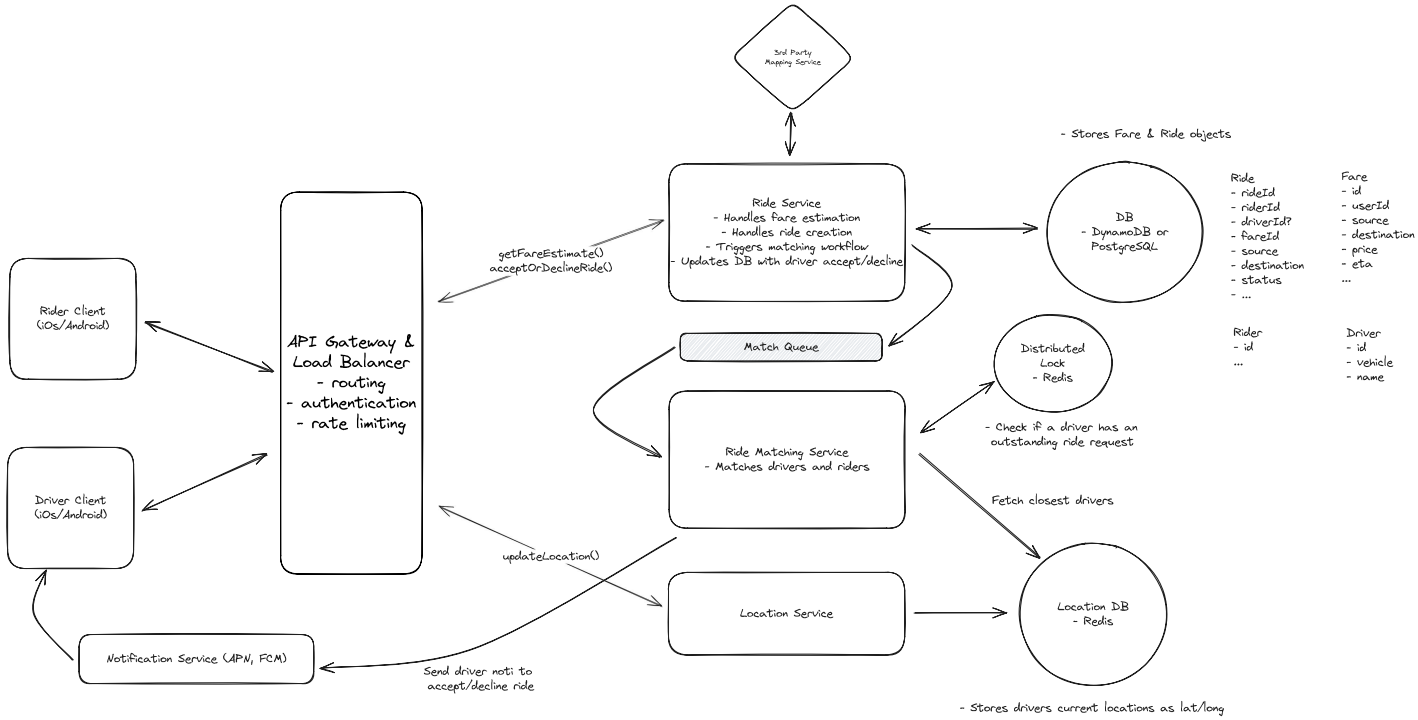
Great Solution: Durable Execution

6) How can you further scale the system to reduce latency and improve throughput?

Bad Solution: Vertical Scaling

Great Solution: Geo-Sharding with Read Replicas

After applying the "Great" solutions, your updated whiteboard should look something like this:



Uber Final

What is Expected at Each Level?

Ok, that was a lot. You may be thinking, "how much of that is actually required from me in an interview?" Let's break it down.

Mid-level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth (80% vs 20%). You should be able to craft a high-level design that meets the functional requirements you've defined, but many of the components will be abstractions with which you only have surface-level familiarity.

Probing the Basics: Your interviewer will spend some time probing the basics to confirm that you know what each component in your system does. For example, if you add an API Gateway, expect that they may ask you what it does and how it works (at a high level). In short, the interviewer is not taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but the interviewer doesn't expect that you are able to proactively recognize problems in your design with high precision. Because of this, it's reasonable that they will take over and drive the later stages of the interview while probing your design.

The Bar for Uber: For this question, an E4 candidate will have clearly defined the API endpoints and data model, landed on a high-level design that is functional and meets the requirements. They would have understood the need for some spatial index to speed up location searches, but may not have landed on a specific solution. They would have also implemented at least the "good solution" for the ride request locking problem.

Senior

Depth of Expertise: As a senior candidate, expectations shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience. It's crucial that you demonstrate a deep understanding of key concepts and technologies relevant to the task at hand.

Advanced System Design: You should be familiar with advanced system design principles. For example, knowing how to use a search-optimized data store like **Elasticsearch** for event searching is essential. You're also expected to understand the use of a distributed cache or similar for locking drivers and to discuss detailed scaling strategies (it's ok if this took some probing/hints from the interviewer), including sharding and replication. Your ability to navigate these advanced topics with confidence and clarity is key.

Articulating Architectural Decisions: You should be able to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for Uber: For this question, E5 candidates are expected to speed through the initial high level design so you can spend time discussing, in detail, at least 2 of the solutions to speed up location searches, the ride request locking problem, or the ride request queueing problem. You should also be able to discuss the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — I'm looking for about 40% breadth and 60% depth in your understanding. This level is all about demonstrating that, while you may not have solved this particular problem before, you have solved enough problems in the real world to be able to confidently design a solution backed by your experience.

You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. The interviewer knows you know the small stuff (REST API, data normalization, etc) so you can breeze through that at a high level so you have time to get into what is interesting.

High Degree of Proactivity: At this level, an exceptional degree of proactivity is expected. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions. Your interviewer should intervene only to focus, not to steer.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making

informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for Uber: For a staff+ candidate, expectations are high regarding depth and quality of solutions, particularly for the complex scenarios discussed earlier. Great candidates are diving deep into at least 3+ key areas, showcasing not just proficiency but also innovative thinking and optimal solution-finding abilities. A crucial indicator of a staff+ candidate's caliber is the level of insight and knowledge they bring to the table. A good measure for this is if the interviewer comes away from the discussion having gained new understanding or perspectives.

Test Your Knowledge

Answer the question below to find your gaps.

Question 1 of 15

Which data structure efficiently partitions 2D space for proximity searches?

1 B-tree

2 Linked list

3 Hash table

4 Quad-tree